Nama:Herbert Batubara

Nim:J0403251162

TPL A2

Praktikum 1

```python
matrix = [
# 0  1  2  3
 [0, 1, 1, 0],  # Baris 0
 [1, 0, 0, 1],  # Baris 1
 [1, 0, 0, 1],  # Baris 2
 [0, 1, 1, 0]   # Baris 3
]

# Tampilkan matrix
print("Adjacency Matrix:")
for row in matrix:
    print(row)

print("\nPenjelasan setiap baris:")

# Penjelasan tiap baris
# Baris 0
print("Baris 0 [0,1,1,0] -> Node 0 terhubung ke node 1 dan 2")

# Baris 1
print("Baris 1 [1,0,0,1] -> Node 1 terhubung ke node 0 dan 3")

# Baris 2
print("Baris 2 [1,0,0,1] -> Node 2 terhubung ke node 0 dan 3")

# Baris 3
print("Baris 3 [0,1,1,0] -> Node 3 terhubung ke node 1 dan 2")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
> & C:/Users/ASUS/AppData/Local/Programs/Python/Python313/pyth
Adjacency Matrix:
[0, 1, 1, 0]
[1, 0, 0, 1]
[1, 0, 0, 1]
[0, 1, 1, 0]

Penjelasan setiap baris:
Baris 0 [0,1,1,0] -> Node 0 terhubung ke node 1 dan 2
Baris 1 [1,0,0,1] -> Node 1 terhubung ke node 0 dan 3
Baris 2 [1,0,0,1] -> Node 2 terhubung ke node 0 dan 3
Baris 3 [0,1,1,0] -> Node 3 terhubung ke node 1 dan 2
PS D:\sem 2\algoritma\Praktikum 11>
```

Praktikum 2

```python
# Graph:
# A --- B
# |     |
# C --- D

# Dictionary digunakan untuk menyimpan adjacency list
# Key = node
# Value = list tetangga (node yang terhubung)
graph = {
    "A": ["B", "C"],  # A terhubung ke B dan C
    "B": ["A", "D"],  # B terhubung ke A dan D
    "C": ["A", "D"],  # C terhubung ke A dan D
    "D": ["B", "C"]   # D terhubung ke B dan C
}

# Menampilkan adjacency list
print("Adjacency List:")
for node, tetangga in graph.items():
    # node = simpul saat ini
    # tetangga = daftar simpul yang terhubung
    print(node, "->", tetangga)

# Penjelasan:
# A -> ['B', 'C'] artinya A terhubung ke B dan C
# B -> ['A', 'D'] artinya B terhubung ke A dan D
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\sem 2\algoritma\Praktikum 11> & C:/Users/ASUS/AppData/Local/Programs/
Adjacency List:
A -> ['B', 'C']
B -> ['A', 'D']
C -> ['A', 'D']
D -> ['B', 'C']
PS D:\sem 2\algoritma\Praktikum 11>
```

Praktikum 3

```python
matrix = [
    [0,1,1,0],  # A
    [1,0,1,0],  # B
    [1,1,0,1],  # C
    [0,0,1,0]   # D
]

# Nama node (sesuai urutan matrix)
nodes = ["A", "B", "C", "D"]

# Dictionary kosong untuk menyimpan hasil adjacency list
graph = {}

# Loop setiap baris (node asal)
for i in range(len(matrix)):
    graph[nodes[i]] = []  # siapkan list kosong

    # Loop setiap kolom (node tujuan)
    for j in range(len(matrix)):
        # Jika bernilai 1 berarti ada hubungan
        if matrix[i][j] == 1:
            # Tambahkan node tujuan ke list
            graph[nodes[i]].append(nodes[j])

# Menampilkan hasil adjacency list
print("Adjacency List hasil konversi:")
for node, tetangga in graph.items():
    print(node, "->", tetangga)

# Penjelasan:
# Baris A [0,1,1,0] -> A terhubung ke B dan C
# Baris B [1,0,1,0] -> B terhubung ke A dan C
# Baris C [1,1,0,1] -> C terhubung ke A, B, dan D
# Baris D [0,0,1,0] -> D terhubung ke C
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Adjacency List hasil konversi:
A -> ['B', 'C']
B -> ['A', 'C']
C -> ['A', 'B', 'D']
D -> ['C']
PS D:\sem 2\algoritma\Praktikum 11>
```

Pertemua 11_Herbert Batubar_J04403251162

```python
graph = {
    "Andi": ["Budi", "Citra"],
    "Budi": ["Deni"],
    "Citra": ["Andi"],
    "Deni": ["Eka"],
    "Eka": ["Budi"]
}

# Tampilkan node
print("Node:")
for i in graph:
    print(i)

# Tampilkan adjacency list
print("\nAdjacency List:")
for i in graph:
    print(i, "->", graph[i])

# Buat adjacency matrix sederhana
nodes = list(graph.keys())
matrix = []

for i in nodes:
    row = []
    for j in nodes:
        if j in graph[i]:
            row.append(1)
        else:
            row.append(0)
    matrix.append(row)

# Tampilkan adjacency matrix
print("\nAdjacency Matrix:")
for row in matrix:
    print(row)

# Tampilkan hubungan
print("\nHubungan:")
for i in graph:
    for j in graph[i]:
        print(i, "follow", j)
```

```
Node:
Andi
Budi
Citra
Deni
Eka

Adjacency List:
Andi -> ['Budi', 'Citra']
Budi -> ['Deni']
Citra -> ['Andi']
Deni -> ['Eka']
Eka -> ['Budi']

Adjacency Matrix:
[0, 1, 1, 0, 0]
[0, 0, 0, 1, 0]
[1, 0, 0, 0, 0]
[0, 0, 0, 0, 1]
[0, 1, 0, 0, 0]

Hubungan:
Andi follow Budi
Andi follow Citra
Budi follow Deni
Citra follow Andi
Deni follow Eka
Eka follow Budi
PS D:\sem 2\algoritma\Praktikum 11>
```

Andi ---> Budi

| |

v v

Citra Deni ---> Eka

^ |

|_____|

Node (Vertex)

Node merepresentasikan pengguna media sosial, yaitu:

- Andi
- Budi
- Citra
- Deni

- Eka

Edge

Edge merepresentasikan hubungan follow antar pengguna, yaitu:

- Andi → Budi
- Andi → Citra
- Budi → Deni
- Citra → Andi
- Deni → Eka
- Eka → Budi

Jumlah:

- Node = 5
- Edge = 6

Analisis Adjacency List vs Matrix

- Adjacency list itu lebih simpel dan hemat, karena cuma nyimpan hubungan yang ada saja. Sedangkan adjacency matrix bentuknya tabel, jadi semua kemungkinan hubungan ditulis, walaupun banyak yang kosong (0).

Jadi untuk media sosial, adjacency list lebih cocok karena tidak semua user saling terhubung.

Kesimpulan

Graph media sosial pakai **directed graph** karena follow itu satu arah. dan graph ini termasuk **sparse**, jadi lebih cocok pakai adjacency list karena lebih efisien.